

Enterprise Information Access Framework (EIAF) - PRD

Enterprise Information Access Framework (EIAF)

Project Requirements Document (PRD)

Version	1.0.0 — MVP Foundation
Author	Sandeep Kumar Mishra
Status	Draft for Development
Audience	Engineering Team

1. Purpose

EIAF is a reusable framework that lets business users ask questions in natural language (voice or text) and safely retrieve information from enterprise databases — without exposing raw schemas, without unrestricted SQL generation, and without giving an LLM direct database access.

This document defines the **MVP scope**, the **layered architecture**, the **input/output contract for every layer**, and the **acceptance criteria** the engineering team must meet for sign-off.

1.1 Long-Term Goal (Context Only — Not in MVP Scope)

The architecture must eventually support four front-end surfaces on one shared backend:

- Voice Interface
- Chat Interface
- Dashboard Interface
- API Interface

1.2 Problem Statement

Traditional architecture (User → LLM → SQL → Database → LLM → Response) is rejected because it is:

- Unsafe (LLM can generate arbitrary SQL)
- Difficult to govern and audit
- Schema-fragile (a column rename breaks prompts)
- Not enterprise-security compliant

1.3 Target Architecture

User → Voice Layer → Intent Layer → Orchestration Layer → Semantic Layer
→ Service Layer → Data Layer → Response Layer → TTS Layer → User

Hard constraint: the LLM is used only to (a) interpret user intent and (b) optionally phrase complex responses in later phases. **It must never receive direct database credentials, raw schema, or the ability to emit SQL that hits production directly.**

2. MVP Scope

2.1 In Scope

--	--

Item	MVP Value
Database	One MySQL database
Table	customers
Business entity	sales
User role	Admin (full access, no restrictions)
Supported question	"How much sales did I do this week?" (and minor phrasing variants — see §4.2)
Interfaces	Text input is mandatory for MVP. Voice input/output (STT/TTS) is a stretch goal — see §2.3

2.2 Out of Scope for MVP

- Multiple tables/entities (Customer, Product, Order)
- Multiple roles, row-level or column-level security
- ClickHouse / analytics warehouse
- Third-party integrations (SAP, Salesforce, Jira, ServiceNow)
- Multi-tool / parallel orchestration
- LLM-based intent classification (rule-based only for MVP)

2.3 MVP Delivery Tiers

To avoid scope ambiguity, MVP delivery is split into two tiers. Confirm with stakeholders which tier defines “done”:

- **Tier 1 (Required):** Text-in → Text-out, full pipeline (Intent → Orchestration → Semantic → Service → Data → Response), Admin role only, `sales.total_sales` metric only.
- **Tier 2 (Stretch):** Voice Layer (STT) and TTS Layer added on top of Tier 1.

2.4 Success Criteria (End-to-End)

Input: "How much sales did I do this week?" **Expected final output:** "This week's total sales are ₹63,000."

Pass condition: the request passes through every layer in §3 with no layer bypassed, and the value returned matches a manual SQL check against the `customers` table.

3. Layer-by-Layer Requirements

Each layer must be an independently deployable/testable module with a defined input and output contract. **No layer may skip the layer before or after it.** Inter-layer communication should use a single shared JSON envelope shape (see §3.0) so layers are swappable later.

3.0 Shared Message Envelope (applies to every layer-to-layer call)

```
{
  "request_id": "uuid",
  "timestamp": "ISO-8601",
  "user_id": "string",
  "role": "string",
  "payload": { },
  "status": "ok | error",
  "error": { "code": "string", "message": "string" }
}
```

`request_id` must be generated once at the entry point and propagated unchanged through all layers — this is the audit/correlation key (see §6.3).

Layer 1 — Voice Layer (Tier 2 / Stretch)

Responsibility: Convert speech to text. **Input:** Audio stream/file. **Output:**

```
{ "text": "How much sales did I do this week?" }
```

Requirements:

- Must emit the shared envelope (\$3.0) with `payload.text` populated.
- On failed/low-confidence transcription, return `status: "error"`, code `STT_LOW_CONFIDENCE`, and do not forward to Layer 2.

Future (not MVP): streaming STT, barge-in, voice activity detection.

Layer 2 — Intent Layer

Responsibility: Determine intent, entities, time filters, parameters from text. **Input:**

```
{ "text": "How much sales did I do this week?" }
```

Output:

```
{ "intent": "sales_amount", "period": "this_week" }
```

MVP Implementation: Rule-based matching (keyword/regex/pattern table), **not** an LLM call. **Requirements:**

- Maintain an explicit, versioned intent-pattern table (file or config, not hardcoded in logic) so patterns can be extended without code changes.
- Unrecognized input → `status: "error"`, code `INTENT_NOT_FOUND`. Do not guess.
- Define the recognized period values for MVP: `today`, `this_week`, `this_month`. Anything else is `INTENT_NOT_FOUND`.

Future: small-LLM-based intent classification.

Layer 3 — Orchestration Layer

Responsibility: Map intent → the business service call to make. **Input:**

```
{ "intent": "sales_amount", "period": "this_week" }
```

Output:

```
{
  "entity": "sales",
  "metric": "total_sales",
  "filters": { "period": "this_week" }
}
```

Requirements:

- For MVP, this is a deterministic lookup table (`intent` → { `entity`, `metric` }), not dynamic reasoning.
- Must validate that the resolved `entity` / `metric` pair actually exists in the Semantic Layer's registry (\$3.4) before forwarding — fail fast with `code: ENTITY_NOT_REGISTERED` rather than letting Layer 4 discover it.

Future: multi-tool and parallel execution, multi-system orchestration.

Layer 4 — Semantic Query Layer

Responsibility: Translate the business request into a validated, SQL-free business query, governed by per-entity config. **Hard rule:** This layer must contain **zero SQL**. It only knows entities, metrics, dimensions, and allowed filters as declared in config.

Example config — semantic/entities/sales.yaml:

```
entity: sales
metrics:
  - total_sales
filters:
  - today
  - this_week
  - this_month
```

Input:

```
{ "entity": "sales", "metric": "total_sales", "filters": { "period": "this_week" } }
```

Output (Validated Business Query):

```
{ "entity": "sales", "metric": "total_sales", "filters": { "period": "this_week" } }
```

Requirements:

- Validate that `metric` is in the entity's declared `metrics` list and `filters.period` is in the entity's declared `filters` list.
- Invalid metric/filter → `code: SEMANTIC_VALIDATION_FAILED`, request stops here (never reaches Service/Data layers).
- Config files must be loaded at startup and hot-reloadable in a later phase (not required for MVP, but don't hardcode the load path in a way that blocks it).

Layer 5 — Service Layer (Security & Governance)

Responsibility: Authn, authz/RBAC, validation, audit logging, rate/query limits.

Input: Validated Business Query (output of Layer 4) + caller identity (`user_id`, `role` from the envelope). **Output:** Approved or Rejected Query (same shape, with `status: ok|error`).

MVP Requirements (explicit, since this is the most security-critical layer):

- **Authentication:** MVP assumes the caller is already authenticated upstream (e.g., session token validated by the API gateway/front-end). The Service Layer trusts `user_id` / `role` in the envelope **only if** they were set by a trusted upstream component — document this trust boundary clearly in code comments and the README.
- **Authorization (RBAC):** Role config example —

```
roles:
  admin:
    allowed_entities: [sales]
```

For MVP only `admin` exists and is allowed `sales`. The check must still be a real lookup against this config, not a bypass — this is what makes adding `manager` / `employee` / `auditor` later a config change, not a code change.

- **Validation:** Re-validate entity/metric/filter against the same semantic config used in Layer 4 (defense in depth — don't trust that Layer 4 was actually called).
- **Audit Logging:** Every request (approved or rejected) must be logged with: `request_id`, `user_id`, `role`, `entity`, `metric`, `filters`, `decision` (approved/rejected), `reason` if rejected, `timestamp`. Logs must be append-only and not user-editable.
- **Query Limits:** Define and enforce a basic rate limit per user (exact number TBD with stakeholders — flagged as an open question in §7) even though MVP has one user role.

Layer 6 — Data Layer

Responsibility: Convert the approved semantic request into SQL using the Schema Registry (§4), execute it, return

raw results.

MVP Database table — customers : | Column | Type (suggested) | Notes | |—|—|—| | customer_name | VARCHAR | | | product | VARCHAR | | | purchase_date | DATE/DATETIME | Used for period filters | | purchase_amount | DECIMAL | Used for total_sales aggregation |

Example generated SQL (for total_sales / this_week):

```
SELECT SUM(purchase_amount)
FROM customers
WHERE purchase_date >= CURRENT_DATE - INTERVAL 7 DAY;
```

Output:

```
{ "total_sales": 63000 }
```

Requirements:

- SQL must be generated exclusively through the Schema Registry mapping (§4) — **never** by string-concatenating user-influenced values directly into SQL. Use parameterized queries throughout to prevent injection, even though filter values come from a controlled enum, not free text.
- Use a read-only DB credential/connection for this layer. The application layer should not have write access to customers for query operations.
- On DB error/timeout, return code: DATA_LAYER_FAILURE — do not let raw DB error messages or stack traces propagate to the Response Layer or end user.
- Define period boundary semantics precisely and confirm with stakeholders (open question, §7): e.g., does “this_week” mean rolling 7 days, or calendar week (Mon–Sun)? The example SQL above uses rolling 7 days — confirm this is correct before building.

Future: MySQL read replicas, ClickHouse, SAP, Salesforce, Jira, ServiceNow.

Layer 7 — Response Layer

Responsibility: Convert raw data into a human-readable response.

Input:

```
{ "total_sales": 63000 }
```

Output:

```
"This week's total sales are ₹63,000."
```

MVP Requirements:

- Template-based only. No LLM call in MVP.
- Template example: "This {period_label} total sales are ₹{amount, formatted with thousands separator}."
- Define period_label mapping (this_week → “week’s”, today → “today’s”, this_month → “month’s”) in config, not inline string logic.
- Currency formatting must use locale-correct thousands separators (₹63,000 — Indian numbering, not 63,000 vs 6,3,000 ambiguity); confirm formatting library choice.

Future: LLM-generated responses for complex analytics.

Layer 8 — Text-to-Speech Layer (Tier 2 / Stretch)

Responsibility: Convert response text to audio. **Input:** "This week's total sales are ₹63,000." **Output:** Audio stream. **Requirements:** Out of scope for Tier 1 acceptance; implement only after Tier 1 sign-off if timeline allows.

4. Schema Registry

Purpose: Decouple business-metric names from physical database structure so schema changes never require changes to Intent, Semantic, Service layers, APIs, or frontend.

Example mapping:

```
total_sales:
  table: customers
  column: purchase_amount
  aggregation: SUM
```

Requirement: If `purchase_amount` is later renamed to `amount`, **only this registry file changes**. This must be enforced by code review: any PR that hardcodes a DB column name outside the Schema Registry / Data Layer should be rejected.

5. Directory Structure (Required Convention)

```
project/
  frontend/
    index.html
    voice.js
  backend/
    main.py
    layers/
      voice/
      intent/
      orchestrator/
      semantic/
      service/
      data/
      response/
      tts/
    semantic/
      entities/
        sales.yaml
    schema_registry/
      mappings.yaml
    database/
      mysql.py
    config/
      roles.yaml
      settings.yaml
```

Each `layers/*` module should be independently unit-testable and expose a single function/class entry point consuming/returning the shared envelope (§3.0).

6. Non-Functional Requirements

6.1 Security

- No layer above the Data Layer may construct or hold raw SQL.
- DB credentials are never exposed to the Intent/Orchestration/Semantic layers.
- All inter-layer payloads should be validated against a schema (e.g., JSON Schema/Pydantic) at each boundary, not just at the edges.

6.2 Extensibility

- Adding a new entity/metric must be achievable via: (1) a new semantic YAML file, (2) a new Schema Registry

mapping, (3) a roles.yaml update — **without modifying Intent, Orchestration, or Service layer code**. This is a hard acceptance criterion, not a nice-to-have — test it explicitly (see §8).

6.3 Auditability

- Every request must be traceable end-to-end via `request_id` across all layer logs.
- Rejected requests must be logged with a reason code, not silently dropped.

6.4 Error Handling

- Every layer must return structured errors via the shared envelope’s `error` object — never raw exceptions/stack traces to the caller.
- Define a shared error-code registry (e.g., `INTENT_NOT_FOUND`, `SEMANTIC_VALIDATION_FAILED`, `ENTITY_NOT_REGISTERED`, `RBAC_DENIED`, `DATA_LAYER_FAILURE`) as a single source of truth all layers reference.

6.5 Testing

- Unit tests per layer using the shared envelope contract as the test boundary (mock the previous/next layer).
- One end-to-end integration test that reproduces the §2.4 success criteria exactly.

7. Open Questions (Must Resolve Before/During Build)

1. **Period boundaries:** Is “this_week” a rolling 7-day window or a calendar week (and which day starts the week)? Same question for “this_month” (rolling 30 days vs. calendar month).
2. **Authentication mechanism:** What issues `user_id` / `role` upstream of the Service Layer for MVP — a simple config flag, a login form, or an existing SSO/session system?
3. **Rate limit values:** What request-per-minute/hour limit should apply, even with a single Admin user?
4. **Tier 2 commitment:** Is voice (STT/TTS) required for MVP sign-off, or is text-only acceptable for v1.0.0 and voice deferred to v1.1?
5. **Hosting/runtime environment:** Where does this run (on-prem, cloud, container) and does that affect the read-only DB credential setup in §3 Layer 6?
6. **Currency/locale:** Is ₹ (INR) the only currency for MVP, or should the Response Layer’s formatting already be locale-aware for future markets?

8. Acceptance Criteria (Definition of Done for MVP)

The MVP is considered complete when **all** of the following are true:

1. Sending the text “How much sales did I do this week?” through the pipeline returns “This week's total sales are ₹63,000.” (or the correct value matching live `customers` data) with no layer bypassed.
2. A minor phrasing variant (e.g., “What were my sales this week?”) is either correctly classified by the Intent Layer or fails clearly with `INTENT_NOT_FOUND` — not silently misclassified.
3. Renaming `purchase_amount` to `amount` in the database, and updating **only** `schema_registry/mappings.yaml`, produces the same correct output with zero other code changes.
4. A request with `role` other than `admin` is rejected at the Service Layer with `RBAC_DENIED`, and this rejection is present in the audit log.
5. An invalid metric (not declared in `sales.yaml`) is rejected at the Semantic Layer with `SEMANTIC_VALIDATION_FAILED` and never reaches the Data Layer.
6. All requests (approved and rejected) appear in the audit log with a consistent `request_id` traceable across every layer’s logs.
7. No SQL string in the codebase is built via direct string concatenation of user-influenced values.

9. Phased Roadmap (Beyond MVP — For Context, Not Current Build)

--	--

Phase	Scope
1	One table, one entity, one metric (this document)
2	Multiple tables/entities — Customer, Product, Order
3	Multiple roles, row-level and column-level security
4	ClickHouse / analytics integration
5	SAP, Salesforce, Jira, ServiceNow integrations
6	Full enterprise voice agent platform

Design principle for all future phases: new business entities should be added primarily through semantic configuration and schema mappings — not by rewriting application/orchestration logic.